KUMULUS

Documentação Técnica da Solução

Assistente Inteligente de Consulta ao Catálogo

Cliente: Rodavale Distribuidora de Autopeças Ltda.

Plataforma: Microsoft Foundry sobre Microsoft Azure

Versão do documento: 1.0

Público: Equipe técnica e responsáveis pela sustentação

Documento técnico de referência. Complementa a Proposta Comercial 2026-014.

1. Visão Geral

> Este documento descreve a arquitetura, os componentes e as decisões técnicas da solução de assistente conversacional implantada para o televendas da Rodavale. Destina-se à equipe de sustentação e a qualquer profissional que precise operar, auditar ou evoluir a solução.

A solução permite que vendedores consultem, em linguagem natural, quais peças do catálogo atendem determinado veículo, qual o saldo em estoque e qual o preço aplicável a cada cliente. Toda resposta é derivada de consulta a dados reais — não há geração de conteúdo sem lastro.

1.1 Componentes

Componente	Serviço Azure	Responsabilidade
Agente	Foundry Agent Service	Interpretação da pergunta, orquestração das consultas e formatação da resposta
Modelo	Foundry Models — GPT-5.4-mini	Compreensão de linguagem natural
Integração	SQL MCP Server (Data API Builder 2.0.12)	Exposição do banco como ferramentas tipadas via Model Context Protocol
Hospedagem	Azure Container Apps	Execução da camada de integração com endpoint público
Imagem	Azure Container Registry	Armazenamento da imagem de container
Dados	Azure SQL Database (Basic)	Catálogo, aplicações, estoque, clientes e preços

1.2 Regiões

Banco de dados, registro de container e ambiente de execução estão provisionados em `brazilsouth`. Essa definição atende ao requisito de residência de dados em território nacional. O projeto Foundry reside em `eastus`, por disponibilidade de modelos; nenhum dado de negócio é persistido nessa região.

2. Arquitetura

2.1 Fluxo de uma consulta

> **1. Entrada**
> O vendedor formula a pergunta em linguagem natural na interface do agente.

2. Interpretação
O agente identifica a intenção, extrai as entidades mencionadas (montadora, modelo, ano, motorização, tipo de peça) e decide quais ferramentas acionar.

3. Descoberta de esquema
Na primeira interação, o agente consulta a estrutura das entidades disponíveis através da ferramenta `describe_entities`.

4. Consulta
O agente aciona `read_records` sobre a entidade adequada, com filtro de igualdade sobre montadora e modelo.

5. Filtragem
Sobre o conjunto retornado, o agente aplica os critérios de ano e motorização, descartando aplicações não pertinentes.

6. Resposta
O resultado é apresentado em tabela, ordenado por preço, com saldo de estoque e observações técnicas de aplicação.

2.2 Comunicação entre camadas

A comunicação entre o Foundry Agent Service e a camada de integração ocorre por **Model Context Protocol**, sobre HTTP. O Agent Service aceita exclusivamente endpoints MCP remotos — por isso a camada de integração é hospedada em Container Apps com ingresso externo, e não executada localmente.

```
Vendedor
   ↓  linguagem natural
Foundry Agent Service  (GPT-5.4-mini)
   ↓  Model Context Protocol / HTTPS
Azure Container Apps  →  SQL MCP Server (Data API Builder)
   ↓  TDS / porta 1433
Azure SQL Database  (rodavale)
```

3. Modelo de Dados

3.1 Tabelas base

Tabela	Chave	Conteúdo
`produtos`	sku	Catálogo de itens: descrição, marca, categoria, posição, preço base e código original da montadora
`aplicacoes`	id	Relação peça–veículo: montadora, modelo, motorização, faixa de anos e observações de aplicação
`estoque`	sku + filial	Saldo, estoque mínimo e localização física por unidade
`clientes`	codigo	Cadastro comercial, com tabela de preço aplicável e limite de crédito
`precos_cliente`	id	Percentual de desconto por combinação de tabela de preço e categoria

> **A tabela central é `aplicacoes`.** É ela que materializa o conhecimento que hoje existe apenas na memória dos vendedores: qual peça atende qual veículo. A qualidade das respostas do assistente é diretamente proporcional à completude desta tabela.

Convenções de preenchimento

- `ano_fim` nulo indica que o veículo permanece em linha; a aplicação é considerada vigente.
- `motorizacao` nula indica que a peça atende todas as versões daquele modelo.
- `observacao` registra restrições de aplicação e é sempre repassada ao vendedor.

3.2 Camada de apresentação — views

Duas views consolidam as regras de negócio e simplificam a consulta pelo agente. Esta é uma decisão de projeto relevante: ao resolver os relacionamentos no banco, o agente executa consultas simples e planas, o que reduz drasticamente a chance de erro na formulação do filtro.

View	Consolida
`vw_pecas_veiculo`	Aplicação, produto e estoque agregado numa única linha. Normaliza motorização nula para `TODAS` e ano final nulo para `9999`.
`vw_preco_cliente`	Cliente, produto e desconto, com o preço final já calculado.

Fórmula de precificação

```
preco_final = preco_base × (1 − desconto_percentual / 100)
```

O desconto é determinado pelo cruzamento entre a `tabela_preco` do cliente (A, B ou C) e a `categoria` do produto.

4. Camada de Integração

4.1 Data API Builder

A exposição do banco é feita pelo Data API Builder, que gera automaticamente endpoints REST e ferramentas MCP a partir de um arquivo de configuração declarativo. Não há código de aplicação proprietário nesta camada — apenas configuração.

Cada tabela ou view declarada no arquivo torna-se uma entidade consultável. Views exigem declaração explícita dos campos que identificam unicamente cada linha, uma vez que não possuem chave primária própria.

4.2 Entidades publicadas

Entidade	Origem	Permissão
PecaVeiculo	`vw_pecas_veiculo`	anonymous: read
PrecoFinal	`vw_preco_cliente`	anonymous: read
Produto	`produtos`	anonymous: read
Aplicacao	`aplicacoes`	anonymous: read
Estoque	`estoque`	anonymous: read
Cliente	`clientes`	anonymous: read
PrecoCliente	`precos_cliente`	anonymous: read

4.3 Ferramentas expostas ao agente

O SQL MCP Server disponibiliza operações tipadas sobre as entidades. O agente não constrói comandos SQL: ele invoca operações com parâmetros estruturados, e a camada de integração traduz para consulta segura. Isso torna cada acesso previsível e auditável.

Ferramenta	Uso
`describe_entities`	Descoberta da estrutura e dos campos disponíveis

`read_records`	Leitura de registros com filtro, seleção de campos e paginação

5. Configuração do Agente

5.1 Modelo

Foi selecionado o **GPT-5.4-mini**. A tarefa envolve interpretação de linguagem, escolha de ferramenta e formatação de resposta — não exige raciocínio complexo. Modelos de maior capacidade elevariam o custo por consulta sem ganho mensurável de qualidade neste caso de uso.

5.2 Instruções

O comportamento do agente é definido por instruções em linguagem natural, não por código. As diretrizes estabelecidas cobrem quatro aspectos:

Diretriz	Objetivo
Consultar antes de perguntar	Evitar que o agente solicite informações desnecessárias. Montadora e modelo são suficientes para iniciar a busca.
Filtros de igualdade apenas	Impedir a construção de expressões compostas, que se mostraram frágeis nos testes.
Nunca inventar dado	Se a consulta não retornar resultado, informar a ausência em vez de gerar conteúdo plausível.
Repassar observações	Restrições de aplicação registradas no cadastro são sempre exibidas ao vendedor.

> **Ajuste de comportamento não exige nova implantação.** As instruções são editadas diretamente no portal e passam a valer na interação seguinte. Isso permite refinamento contínuo a partir do uso real, sem ciclo de desenvolvimento.

6. Segurança

6.1 Controle de acesso ao banco

O servidor SQL possui duas formas de autenticação configuradas: autenticação SQL, utilizada pela camada de integração, e Microsoft Entra ID, destinada ao acesso administrativo por pessoas. O firewall restringe conexões a endereços autorizados e a serviços do próprio Azure.

6.2 Restrição a leitura

Todas as entidades são publicadas exclusivamente com permissão de leitura. Não existe caminho técnico pelo qual o assistente possa inserir, alterar ou excluir registros. A operação transacional permanece integralmente no ERP.

6.3 Tratamento de credenciais

A cadeia de conexão com o banco não está embutida na imagem de container. É injetada como variável de ambiente na configuração do Container App, e a senha do registro de container é armazenada como segredo gerenciado pela plataforma.

6.4 Rastreabilidade

A plataforma Foundry registra cada interação: pergunta recebida, ferramentas acionadas, parâmetros utilizados e resposta gerada. Esses registros permitem auditoria posterior e análise de qualidade das respostas.

> **Ponto de atenção para produção.** Na configuração atual, o endpoint da camada de integração é público e não autenticado, adequado ao contexto de validação. Para operação em produção, recomenda-se autenticação por identidade gerenciada e restrição de rede, conforme descrito na seção 8.

7. Operação

7.1 Atualização do catálogo

Na configuração atual, a carga de dados é executada manualmente a partir de extração do ERP. A automação dessa sincronização está prevista como evolução.

7.2 Alteração de entidades expostas

Incluir uma nova tabela ou view no escopo de consulta do agente requer: declaração da entidade na configuração, reconstrução da imagem de container e atualização do Container App. O ciclo completo leva poucos minutos e não implica indisponibilidade, uma vez que a plataforma realiza a substituição por revisões.

7.3 Monitoramento

O que observar	Onde
Erros de execução da camada de integração	Logs do Container App (Log Analytics)
Interações do agente e ferramentas acionadas	Rastreamento no portal Foundry
Consumo de tokens e custo	Painel de uso do projeto Foundry
Desempenho e conexões do banco	Métricas do Azure SQL Database

8. Limitações Conhecidas

Registro transparente das restrições da configuração atual e do encaminhamento previsto para cada uma.

Limitação	Impacto	Encaminhamento
Endpoint de integração público e sem autenticação	Terceiro com conhecimento da URL poderia consultar o catálogo	Autenticação por identidade gerenciada e restrição de rede
Carga de dados manual	Divergência entre ERP e base de consulta entre cargas	Rotina de sincronização automatizada
Ausência de tratamento de erro no agente	Falha na consulta resulta em mensagem técnica ao vendedor	Instrução de fallback e mensagem amigável
Consumo elevado de tokens por consulta	Custo proporcionalmente maior em alto volume	Redução do contexto e cache de descoberta de esquema
Base de demonstração é amostra do catálogo	Cobertura parcial de veículos e peças	Carga completa na implantação

> **Sobre o consumo de tokens.** As medições realizadas indicam consumo aproximado de 17 mil tokens por consulta, valor influenciado pela descoberta de esquema e pelo volume de registros retornados. Estratégias de otimização — restrição de campos selecionados e limitação do conjunto de resultados — devem ser avaliadas após observação do padrão de uso real.

9. Evolução Técnica

Etapa	Descrição técnica
Sincronização automatizada	Rotina agendada de extração e carga incremental a partir do ERP, com controle de versão dos registros alterados.
Autenticação da integração	Substituição do acesso anônimo por identidade gerenciada do projeto Foundry, com o Container App validando o token emitido.
Isolamento de rede	Implantação do Container App em sub-rede dedicada, com ingresso interno e conexão privada ao banco.
Canal para o cliente final	Exposição do agente via WhatsApp, com camada de identificação do cliente e restrição das entidades acessíveis.

Telemetria de negócio	Registro estruturado das consultas para identificar demanda por itens ausentes do catálogo ou sem estoque.

10. Referências

- Microsoft Foundry — documentação do Agent Service e de ferramentas MCP
- Data API Builder — configuração de entidades e SQL MCP Server
- Azure Container Apps — implantação e gestão de revisões
- Azure SQL Database — segurança, firewall e autenticação

Kumulus · Documentação Técnica v1.0 · Rodavale Distribuidora de Autopeças
Documento confidencial — reprodução não autorizada